

Technische Universität Chemnitz

Exploring Reinforcement learning strategies in automated wheel truing

Master's Thesis

Toni Rozsahegyi

First Supervisor: M.Sc. Antje Ahrens

Second Supervisor: M.Sc. Oliver Maith

last updated on: January 6, 2026

Abstract

This thesis attempts to find a solution to automatic spoked wheel truing through reinforcement learning in a simulated environment. We test and evaluate a set of different algorithms, state-spaces and reward schemes.

Contents

Abstract	i
1 Introduction	1
2 Theory	3
2.1 Foundations of Reinforcement Learning	3
2.1.1 The Markov Decision Process Framework	3
2.1.2 Evaluating Policies through Value Functions	4
2.1.3 Finding Optimal Policies	4
2.1.4 The Exploration-Exploitation Dilemma	5
2.1.5 The Curse of Dimensionality	6
2.2 Deep Reinforcement Learning	6
2.2.1 Challenges of Function Approximation	6
2.2.2 Three Algorithmic Paradigms	7
2.3 Value-Based Methods	8
2.3.1 Stabilization Techniques	8
2.3.2 Distributional Reinforcement Learning and Exploration	9
2.4 Policy Gradient Methods	9
2.4.1 The Policy Gradient Theorem	9
2.4.2 Variance Reduction through Advantage Estimation	10
2.4.3 Trust Regions and Proximal Policy Optimization	10
2.5 Model-Based Methods	11
2.5.1 World Models and Latent Representations	11
2.5.2 Planning with Learned Models	11
2.5.3 TD-MPC2	12
3 Methods	13
3.1 Environment	13
3.1.1 Wheel Truing Physics	13
3.1.2 On State-Space Formulations	20
3.1.3 On the Starting State	21

3.1.4	On Action-Space Formulations	21
3.1.5	On Reward Functions	24
3.2	Algorithm Configurations	25
3.2.1	Rainbow DQN	26
3.2.2	PPO	29
3.2.3	TD-MPC2	32
3.2.4	Common Training Configuration	35
3.3	Experimental Design	36
3.3.1	Ablation Study Design	36
3.3.2	Methodological Limitations	37
4	Results and Analysis	38
4.1	Baseline Performance	38
4.1.1	distribution of metrics at reset	38
4.1.2	DQN	39
4.1.3	PPO	40
4.1.4	TD-MPC2	41
5	Discussion and Conclusion	43
	Bibliography	44
	Appendix	45
.1	Code Listings	45

List of Figures

2.1	The policy iteration algorithm alternates between evaluating the current policy and improving it by acting greedily with respect to the computed values. This process converges to the optimal value function V^* and optimal policy π^*	5
3.1	Distribution of initial state components across 500 sampled wheel configurations. (a) Radial displacements are approximately an order of magnitude smaller than (b) lateral displacements. (c) Spoke tensions are normalized by the initial tension $T_0 = 800$ N, centering the distribution around 1.0. (d) The Fourier magnitude spectrum shows that lower harmonics dominate the initial disturbances. Red lines indicate mean and ± 1 standard deviation.	22
4.1	initial state norm distribution.	38
4.2	initial maximum tension deviation (from 800).	39
4.3	Rainbow	39
4.4	PPO continuous	40
4.5	PPO discrete	40
4.6	PPO all spokes	41
4.7	TDMPC all spokes	41
4.8	TDMPC continuous	42

List of Tables

3.1	Wheel simulation parameters	20
3.2	Algorithm–action space compatibility matrix. Checkmarks indicate valid combinations tested in this work.	23
3.3	Rainbow DQN hyperparameters	28
3.4	PPO hyperparameters	31
3.5	TD-MPC2 network architectures. All hidden layers use Mish activation with layer normalization.	32
3.6	TD-MPC2 planning hyperparameters	34
3.7	TD-MPC2 training hyperparameters	35
3.8	Shared training configuration across all algorithms	35

Chapter 1

Introduction

Broader Motivation Many industrial manufacturing processes produce components that retain internal stresses from thermal or mechanical treatment. These residual stresses pose significant challenges in sectors where large monolithic structures are employed, such as shipbuilding and aircraft manufacturing. If left unaddressed, internal tensions can lead to gradual deformation, reduced fatigue life, or catastrophic material failure during operation.

In shipbuilding, for instance, large hull sections and structural components that have undergone welding or thermal cutting often require stress relief before assembly. This remains a predominantly manual task, demanding considerable expertise. Skilled specialists iteratively apply localized heating to specific regions of the workpiece, allowing the material to settle into a lower-energy configuration upon cooling. The process is inherently sequential: the technician observes the current state of the structure, decides where to apply heat, and evaluates the result before proceeding to the next intervention. This cycle continues until the component reaches an acceptable stress distribution.

While the physical mechanisms differ substantially, the wheel truing problem studied in this thesis shares conceptual similarities with such stress optimization tasks. Both involve iteratively adjusting tension within a coupled system, where local interventions propagate effects throughout the structure. Both require balancing competing constraints—in wheel truing, achieving geometric trueness while maintaining uniform spoke tensions; in stress relief, minimizing residual stresses without introducing new distortions. These parallels suggest that methods developed for automated wheel truing may offer insights applicable to a broader class of iterative tension optimization problems.

Wheel Truing A spoked bicycle wheel consists of a hub connected to a rim through a network of tensioned spokes. During use or as a result of manufacturing imperfections, the rim can deviate from its ideal circular and planar geometry a condition known as being "out of true." Wheel truing is the process of restoring geometric accuracy by selectively adjusting spoke tensions. Tightening or loosening individual spokes pulls the

rim in specific directions, and through careful iterative adjustments, a skilled mechanic can correct both lateral (side-to-side) and radial (up-and-down) deviations.

This task presents an interesting challenge for reinforcement learning. The state space is high-dimensional, encoding the displacement field around the rim and the tensions of all spokes. Actions are discrete choices of which spoke to adjust and by how much, yet their effects propagate non-locally through the coupled spoke-rim system. The reward signal must balance multiple objectives: minimizing rim displacement, maintaining spoke tensions within safe bounds, and achieving uniform tension distribution. These characteristics make wheel truing a compelling testbed for studying sequential decision-making in physically grounded environments.

The longer-term goal of this research program, though beyond the scope of the present thesis, is to achieve sim-to-real transfer: training an agent entirely in simulation that can subsequently be deployed on a physical wheel truing machine. Such a system would iteratively measure the wheel’s geometry, select a spoke to adjust, execute the adjustment, and repeat until the wheel meets trueness specifications. Automated wheel truing machines do exist commercially, but to our knowledge, current systems rely on inverse models calibrated to specific wheel configurations and do not generalize across different wheel geometries or spoke patterns. A reinforcement learning approach trained on diverse simulated wheels could potentially overcome this limitation, learning policies that transfer to novel wheel types without explicit re-calibration.

This horizon goal informs several design decisions throughout this work. The choice of state representations considers what measurements would be feasible on a physical truing stand. The action space formulations reflect realistic constraints on spoke adjustment granularity.

Chapter 2

Theory

2.1 Foundations of Reinforcement Learning

Reinforcement learning addresses the problem of sequential decision-making under uncertainty. Unlike supervised learning, where a model learns from labeled examples, a reinforcement learning agent must discover effective behavior through interaction with its environment. The agent receives no direct instruction on which actions to take; instead, it observes the consequences of its actions through a scalar reward signal. This feedback is often delayed, a decision made now may only reveal its quality many steps later. Furthermore, the agent's actions influence not only immediate rewards but also the future states it will encounter.

2.1.1 The Markov Decision Process Framework

The standard formalization of this problem is the Markov decision process (MDP). An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, p, r)$, where $\mathcal{S}$ denotes the set of possible states, $\mathcal{A}$ the set of available actions, $p(s' | s, a)$ the probability of transitioning to the next state s' when taking action a in state s , and $r(s, a, s')$ the reward received upon this transition. The defining characteristic of this framework is the Markov property: the distribution over next states depends only on the current state and action, not on the history of how that state was reached. This memorylessness is what makes the problem tractable and allows for mathematical rigor of the underlying optimization procedure. All information relevant to future outcomes is encoded in the current state, allowing the agent to make decisions without maintaining an explicit record of the past.

The agent's behavior is governed by a policy π , which specifies a probability distribution over actions for each state: $\pi(a | s) = P(a_t = a | s_t = s)$. A policy can be deterministic, always selecting the same action in a given state, or stochastic, randomizing over multiple actions. The central objective of reinforcement learning is to find a policy that maximizes the cumulative reward the agent receives over time.

2.1.2 Evaluating Policies through Value Functions

To compare policies, we need a way to quantify their long-term performance. The discounted return starting from time t is defined as

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (2.1)$$

where $\gamma \in [0, 1)$ is the discount factor. This formulation captures two important intuitions. First, rewards received sooner are generally preferable to the same rewards received later—a principle familiar from economics. Second, the discount factor ensures that the infinite sum remains bounded, which is necessary for the problem to be well-defined.

The state-value function $V^\pi(s)$ measures the expected return when starting in state s and subsequently following policy π :

$$V^\pi(s) = \mathbb{E}_\pi[R_t \mid s_t = s]. \quad (2.2)$$

Similarly, the action-value function $Q^\pi(s, a)$ captures the expected return when taking action a in state s and then following π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[R_t \mid s_t = s, a_t = a]. \quad (2.3)$$

These two functions are intimately related: the state-value is the expectation of the action-value under the policy, $V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$.

A crucial property of value functions is their recursive structure, expressed by the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')]. \quad (2.4)$$

This equation states that the value of a state equals the expected immediate reward plus the discounted value of the successor state. The recursive nature of this relationship forms the foundation for reinforcement learning algorithms.

2.1.3 Finding Optimal Policies

Among all possible policies, at least one achieves the maximum expected return from every state. The corresponding optimal value functions are defined as

$$V^*(s) = \max_{\pi} V^\pi(s), \quad Q^*(s, a) = \max_{\pi} Q^\pi(s, a). \quad (2.5)$$

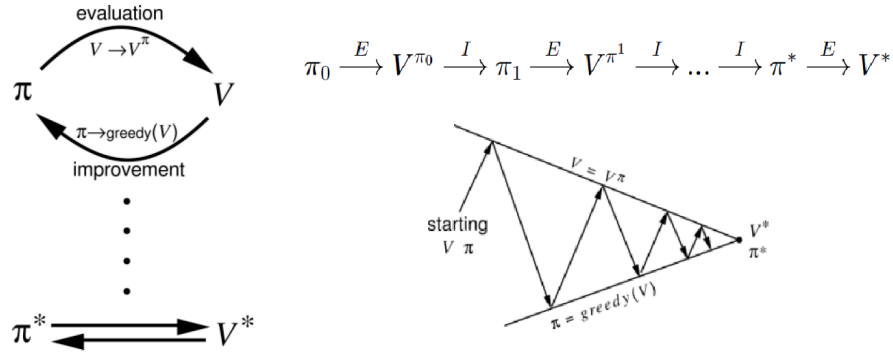


Figure 2.1: The policy iteration algorithm alternates between evaluating the current policy and improving it by acting greedily with respect to the computed values. This process converges to the optimal value function V^* and optimal policy π^* .

If the optimal action-value function Q^* were known, the optimal policy would follow immediately: simply select the action with the highest Q -value in each state. The challenge, of course, is that Q^* is not given—it must be learned.

The classical approach to finding optimal policies is policy iteration, which alternates between two phases. In policy evaluation, the value function V^π is computed for the current policy by solving the Bellman equation. In policy improvement, a new policy is derived by acting greedily with respect to the computed values: selecting the action that maximizes expected return according to V^π . This improved policy is then evaluated, and the cycle repeats. Under mild conditions, this process converges to the optimal policy. Figure 2.1 illustrates this iterative refinement, showing how the value function progressively improves until reaching V^* .

2.1.4 The Exploration-Exploitation Dilemma

A fundamental tension pervades all of reinforcement learning: the conflict between exploration and exploitation. Exploitation means selecting actions that the agent currently believes to be best, based on its learned value estimates. Exploration means trying actions whose values are uncertain, potentially discovering better strategies. An agent that exploits too aggressively may settle for mediocre policies, never discovering superior alternatives. An agent that explores too freely wastes time on suboptimal actions even when good ones are known.

This dilemma has no universal solution. Different algorithms adopt different strategies—from simple approaches like ϵ -greedy action selection, which occasionally chooses random actions, to sophisticated methods that quantify and seek out uncertainty. The appropriate balance depends on the problem structure and the available learning budget. What remains constant is that effective learning requires both: pure exploitation leads to premature convergence, while pure exploration never capitalizes on acquired knowledge.

2.1.5 The Curse of Dimensionality

The algorithms described so far assume that value functions can be represented exactly for instance, as tables storing one value per state or state-action pair. This tabular approach works well for small problems with discrete states. A gridworld with a hundred cells requires only a hundred table entries; policy iteration can solve such problems efficiently.

bicycle wheel with continuous spoke tensions and rim displacements has an effectively infinite state space. The number of entries required for a tabular representation grows exponentially with the dimensionality of the state a phenomenon known as the curse of dimensionality. Exact methods become computationally infeasible long before reaching problems of practical interest.

This limitation motivates the use of function approximation: rather than storing values explicitly for each state, we approximate the value function with a parameterized model. Under the assumption that a suitable function from states to "true" values exists, this model generalizes from visited states to unvisited ones, enabling learning in vast state spaces. As universal function appropriators, neural networks have proven particularly effective for this purpose, capable of representing complex, nonlinear relationships between these states and values. This combination of reinforcement learning with neural network function approximation defines the field of deep reinforcement learning.

2.2 Deep Reinforcement Learning

Replacing tables with neural networks fundamentally changes the learning problem. Function approximation introduces challenges absent from the tabular setting, and much of modern deep reinforcement learning research concerns methods for overcoming these difficulties.

2.2.1 Challenges of Function Approximation

Three interrelated problems arise when neural networks approximate value functions or policies.

The first is non-stationarity of the learning target. In supervised learning, the relationship between inputs and outputs is fixed: a cat remains a cat regardless of how well the classifier performs. In reinforcement learning, the target values depend on the current policy, which itself changes as learning progresses. When approximating Q^π , the "true" values shift as π improves. The network chases a moving target, and updates that improve accuracy for one policy may be counterproductive for the next.

The second challenge stems from correlated samples. Standard supervised learning assumes that training examples are independent and identically distributed. Experience

collected by an agent violates this assumption profoundly: consecutive states in a trajectory are highly correlated, and the distribution of visited states depends on the current policy. Training on sequential batches of correlated experience can cause the network to overfit to recent states, forgetting what it learned earlier.

The third problem is bootstrapping instability. Many reinforcement learning algorithms update value estimates using other value estimates—the Bellman equation, for instance, expresses $V(s)$ in terms of $V(s')$. When the same network provides both the prediction and the target, small errors can amplify: an overestimated value propagates to predecessor states, which then propagate further, potentially leading to divergence.

These challenges are not independent. Non-stationarity exacerbates instability; correlated samples accelerate overfitting to erroneous values. Effective deep reinforcement learning algorithms must address all three.

The correlation problem admits a universal solution applicable across algorithmic paradigms: experience replay. Rather than training on transitions as they occur, the agent stores experiences in a replay buffer and samples mini-batches randomly for training. This breaks temporal correlations and allows each transition to contribute to multiple updates. Prioritized variants improve efficiency further by sampling transitions proportionally to their prediction error, focusing computation on experiences the network currently models poorly. Experience replay has become a standard component in both value-based and model-based methods, and similar buffer-based approaches appear in some policy gradient implementations.

The remaining challenges—non-stationarity and bootstrapping instability—are addressed differently by each algorithmic paradigm. Value-based methods employ target networks and double estimation to stabilize the bootstrap targets. Policy gradient methods sidestep bootstrapping entirely by estimating returns from complete trajectories, and constrain policy changes to maintain stability despite the shifting objective. Model-based methods learn an explicit dynamics model, allowing them to generate synthetic experience and plan ahead rather than relying solely on cached value estimates. The following sections examine each approach in detail.

2.2.2 Three Algorithmic Paradigms

Deep reinforcement learning algorithms can be broadly categorized into three families based on what they learn and how they derive behavior.

Value-based methods learn an approximation to the optimal action-value function Q^* . The policy is implicit: in each state, the agent selects the action with the highest estimated Q-value. Because this requires evaluating Q for every possible action, value-based methods are most naturally suited to discrete action spaces where actions can be enumerated.

Policy gradient methods parameterize the policy directly as $\pi_\theta(a \mid s)$ and optimize the expected return with respect to the policy parameters θ . Note that this θ is unrelated to the variable used in the wheel state. This approach handles continuous action spaces naturally: the network outputs parameters of a continuous distribution, such as the mean and variance of a Gaussian. The policy can be stochastic, which provides a natural mechanism for exploration.

Model-based methods learn a model of the environment’s dynamics: given the current state and action, predict the next state and reward. This model enables planning by simulating the consequences of actions before committing to them. By extracting more information from each real interaction, model-based approaches can achieve higher sample efficiency than their model-free counterparts.

The following sections examine each paradigm in greater detail, focusing on the mechanisms that address the challenges outlined above.

2.3 Value-Based Methods

Value-based methods seek to learn the optimal action-value function $Q^*(s, a)$ directly. Once learned, the optimal policy follows immediately: select $\arg \max_a Q^*(s, a)$ in each state. The foundational algorithm in this family is Q-learning, which updates value estimates using the Bellman optimality equation. Deep Q-Networks (DQN) extend this approach using neural networks as function approximators, introducing several mechanisms to stabilize training.

2.3.1 Stabilization Techniques

Target networks tackle bootstrapping instability by separating the network used for action selection from the network used to compute target values. The target network is a copy of the main network, updated only periodically—for instance, every several thousand steps. This separation prevents the feedback loop where changes to the network immediately affect its own training targets, providing a more stable learning signal.

Double Q-learning addresses a systematic overestimation bias in standard Q-learning. Because the same network both selects the best action and evaluates its value, noise in the estimates tends to favor overestimated actions. Double Q-learning decouples these roles: the main network selects the action, but the target network evaluates it. This simple modification significantly reduces overestimation and improves learning stability.

2.3.2 Distributional Reinforcement Learning and Exploration

Standard value functions represent expected returns—a single number summarizing the distribution of possible outcomes. Distributional reinforcement learning instead learns the full distribution of returns. This richer representation captures information about risk and uncertainty that expectations discard. Empirically, distributional methods often learn faster and achieve better final performance, even when the distribution itself is ultimately reduced to an expectation for action selection.

Implicit Quantile Networks (IQN) provide a flexible approach to distributional learning. Rather than predicting a fixed set of quantiles, IQN learns to map any quantile fraction $\tau \in [0, 1]$ to its corresponding return value. This implicit representation adapts to the shape of the return distribution without requiring manual specification of its support.

Exploration in value-based methods traditionally relies on ε -greedy action selection: with probability ε , choose a random action; otherwise, act greedily. Noisy networks offer a learned alternative by adding parameterized noise to the network weights. The magnitude and structure of the noise adapt during training, providing state-dependent exploration that can be more efficient than uniform random action selection.

The Rainbow algorithm combines these advances—experience replay, target networks, double Q-learning, distributional values, noisy exploration, and several others—into a unified framework. The specific configuration for this and the other two algorithms employed in this work is detailed in the later Methods section.

2.4 Policy Gradient Methods

Policy gradient methods take a fundamentally different approach: rather than learning values and deriving a policy indirectly, they parameterize the policy $\pi_\theta(a \mid s)$ and optimize it directly. This formulation handles continuous action spaces naturally. The network outputs the parameters of a probability distribution—for instance, the mean and variance of a Gaussian—from which actions are sampled. Stochasticity is intrinsic to the policy rather than added externally for exploration.

2.4.1 The Policy Gradient Theorem

The objective is to maximize the expected return under the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R_0]. \quad (2.6)$$

The policy gradient theorem provides a formula for the gradient of this objective:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R_t]. \quad (2.7)$$

This expression can be estimated from sampled trajectories without requiring knowledge of the environment dynamics. Actions that lead to high returns are reinforced; their probabilities increase. Actions leading to low returns are suppressed.

The main challenge with this estimator is variance. Returns can fluctuate dramatically between episodes, and multiplying log-probabilities by these noisy signals produces gradient estimates that vary wildly. High variance slows learning and can destabilize training.

2.4.2 Variance Reduction through Advantage Estimation

A key insight is that the gradient formula remains valid if a state-dependent baseline $b(s_t)$ is subtracted from the return:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (R_t - b(s_t))]. \quad (2.8)$$

If the baseline approximates the expected return $V^{\pi}(s_t)$, the quantity $R_t - b(s_t)$ estimates the advantage $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$: how much better action a_t was compared to the average action in state s_t . Subtracting the baseline does not change the expected gradient but substantially reduces its variance.

Generalized Advantage Estimation (GAE) provides a practical method for computing advantages that balances bias and variance through a parameter λ . Low values of λ yield low-variance but potentially biased estimates; high values reduce bias at the cost of increased variance. This tunability allows practitioners to adapt the algorithm to the characteristics of their problem.

2.4.3 Trust Regions and Proximal Policy Optimization

Policy gradient updates face a different danger: because the gradient is estimated using samples from the current policy, the estimate becomes unreliable if the policy changes substantially. A large update might move the policy far from the region where the gradient estimate is accurate, potentially causing catastrophic performance collapse.

Trust region methods address this by constraining how much the policy can change in a single update. The idea is to improve the policy while staying close to the current one, measured by a divergence such as the Kullback-Leibler divergence. Trust Region Policy Optimization (TRPO) enforces this constraint explicitly but requires computationally expensive second-order optimization.

Proximal Policy Optimization (PPO) achieves similar stability through a simpler mechanism. It tracks the probability ratio between the updated and original policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (2.9)$$

The objective function clips this ratio to the interval $[1 - \epsilon, 1 + \epsilon]$, preventing large policy changes regardless of how advantageous they might appear. This clipping serves as a soft trust region: updates that would change the policy too much are truncated. The resulting algorithm is simple to implement, requires only first-order optimization, and has proven remarkably effective across a wide range of problems.

2.5 Model-Based Methods

Model-based reinforcement learning learns a model of the environment—predicting how states evolve and what rewards result from actions—and uses this model to inform decision-making. While model-free methods learn to act well through trial and error, model-based methods learn how the world works and leverage this understanding for more efficient learning.

2.5.1 World Models and Latent Representations

Learning to predict raw observations directly can be challenging, particularly for high-dimensional inputs. World models address this by learning a compact latent representation of states together with dynamics that operate in this latent space. An encoder maps observations to latent states; a dynamics model predicts how latent states evolve under actions; and auxiliary networks predict rewards and values from latent states.

Operating in latent space offers several advantages. The representation can discard irrelevant details, focusing on aspects of the state that matter for decision-making. Dynamics in latent space are often smoother and more predictable than in observation space. And the compact representation reduces the computational burden of both learning and planning.

2.5.2 Planning with Learned Models

Given a learned model, the agent can simulate the consequences of candidate action sequences without actually executing them. Model Predictive Control (MPC) optimizes actions over a finite planning horizon: starting from the current state, the model predicts trajectories for many candidate action sequences, evaluates their expected returns, and

selects the first action of the best trajectory. After executing this action and observing the new state, planning repeats from scratch.

Re-planning at every step provides robustness to model errors. Even if predictions drift from reality over long horizons, the agent corrects course before errors accumulate catastrophically. Combining model-based planning with a learned value function further helps: rather than predicting rewards over an infinite horizon, the planner need only look a few steps ahead, using the value function to estimate the worth of terminal states.

2.5.3 TD-MPC2

TD-MPC2 exemplifies this hybrid approach. It jointly learns five components: an encoder mapping observations to latent states, a dynamics model predicting latent state transitions, a reward model predicting immediate rewards, a policy network suggesting promising actions, and a value network estimating long-term returns. At decision time, the algorithm uses the policy network to propose candidate action sequences, refines them through model-based trajectory optimization, and executes the best action found.

This architecture balances the sample efficiency of model-based learning with the robustness of model-free value estimation. The learned model enables efficient planning, while the value function limits the horizon over which the model must remain accurate.

Chapter 3

Methods

TODO:

- introduce the thinking behind different reward functions
- make sure the wheel parameters have the right units and descriptions (sorta done needs last look)
- show plot and/or explain elbow graph
- improve on the reasoning for stopping condition
- reason why 41
- use figures to show the network structure and value/policy approximation functions / gradient flow or learning
- finish physical simulation section

3.1 Environment

3.1.1 Wheel Truing Physics

The environment used in this thesis is built upon a simulation based on a physical model of spoked bicycle wheels presented in [2]. This section covers a brief overview of the problem formulation and numerical solution to the spoked wheel system.

Structural components and modeling assumptions

The spoked wheel is essentially built from three different components: the hub, rim and a number of spokes. Relative to the spokes and rim, the hub is a rigid structure of high stiffness; it is therefore modeled as stationary and non-deformable and provides the boundary condition of the spoke-rim system through its diameter d_h and width w_h . The

rim is in essence a thin-walled, curved beam with constant radius R measured at the shear center of the cross-section. This cross-section features a complex cross-sectional geometry exhibiting monosymmetric geometry (symmetric about the radial axis but not the lateral axis) which introduces coupling between lateral bending and torsional deformations. To describe these interactions, the rim is modeled as being made from a linearly elastic material obeying the generalized Hooke's law, which forms a beam obeying Euler-Bernoulli theory. This boils down to neglecting shear deformations that are not associated with uniform torsion (plane sections remain plane and perpendicular to the neutral axis during bending) and ignoring in-plane distortion of the rim cross-section. From Hooke's law we need the Young's and shear moduli E and G respectively. The spokes are modeled as ideal, linear elastic bar elements, deforming only along their length with a Young's modulus E_s . The spokes are connected to the rim and hub by idealized, moment-free, ball-and-socket joints, meaning they do not transmit bending moments or torques. To describe deformations in the wheel, a local cylindrical coordinate system is defined through a right-handed orthonormal triad. The basis vectors $\{e_1, e_2, e_3\}$ of this triad point in lateral (out of plane), radial (toward hub) and tangential (increasing arc length) directions. The origin travels with the rim centroid specified by an angular coordinate $\theta \in [-\pi, \pi]$. With this system we can describe the spoke position through θ and directions through direction cosines $\{c_1, c_2, c_3\}$ defined by the spoke unit vector $n = c_1 e_1 + c_2 e_2 + c_3 e_3$ with $c_1^2 + c_2^2 + c_3^2 = 1$.

Displacement Field

The displacement of the rim is described by its motion and rotation of the shear center. The displacement of the shear center is expressed in the local coordinate basis:

- $u(\theta)$: Lateral displacement in e_1 direction (parallel to hub axis)
- $v(\theta)$: Radial displacement in e_2 direction (positive inward)
- $w(\theta)$: Tangential displacement in e_3 direction (along rim circumference)

The rotation is characterized by a single angle:

- $\Phi(\theta)$: Twist about e_3 axis (normal to cross-section)

These components collect into an augmented displacement vector:

$$d = [u, v, w, \phi]^T \quad (3.1)$$

Which is the fundamental unknown of the governing equations. At any arbitrary point, the displacement field can be obtained by adding the rigid-body rotation and warping contributions:

$$u(\theta, x, y) = u_s + \omega \times r + \frac{\phi'}{R} \alpha_s e_3 \quad (3.2)$$

where u_s is the shear center displacement, ω is the rotation vector, $r = [x, y - y_0, 0]^T$ is the position vector from the shear center to the point of interest, α_s is the warping function, and y_0 is the offset between the centroid and shear center.

Strain Energy

The strain energy stored in the rim due to deformation provides the starting point for deriving the equations of motion. By Hamilton's principle or the principle of virtual work, the equilibrium configuration minimizes the total potential energy (strain energy minus work done by external forces). The infinitesimal rotation angles of the basis vectors, to first order, are:

$$\omega_1 = v' + \frac{\omega}{R}, \omega_2 = u', \omega_3 = \phi \quad (3.3)$$

where primes denote differentiation with respect to arc length. The curvature changes associated with bending and twisting are:

$$\kappa_1 = \frac{u'}{R} = \frac{1}{R} \frac{du}{d\theta}, \kappa_2 = -\frac{v'}{R} = -\frac{1}{R} \frac{dv}{d\theta}, \kappa_3 = \frac{\phi'}{R} = \frac{1}{R} \frac{d\phi}{d\theta} \quad (3.4)$$

The longitudinal strain at a point in the cross-section is given by:

$$\varepsilon_{33} = \varepsilon_c - x\kappa_1 - y\kappa_2 + \alpha_s \kappa_3 \quad (3.5)$$

where ε_c is the strain at the centroid. The first two terms represent the contribution from bending (with the familiar linear variation across the section), while the last term accounts for the effect of non-uniform torsion (warping). The shear strain from uniform torsion is:

$$\gamma = r(\phi' - \frac{u'}{R}) \quad (3.6)$$

where r is the distance from the shear center. This strain is responsible for the torsional stiffness of the rim. The strain energy is obtained by integrating the strain energy density $\frac{1}{2}(E\varepsilon_{33}^2 + G\gamma^2)$ over the rim volume:

$$U_{rim} = \frac{1}{2} \int_0^{2\pi R} [EA\varepsilon_c^2 + EI_1\kappa_1^2 + EI_2\kappa_2^2 + GJ\kappa_3^2 + EI_\omega(\kappa_3')^2] ds \quad (3.7)$$

This result is obtained by substituting the strain expressions, expanding the squares, and using the definitions of the cross-sectional properties:

$$A = \iint dA, I_1 = \iint y^2 dA, I_2 = \iint x^2 dA, J = \iint 4\xi^2 dA, I_{\omega} = \iint \alpha_s^2 dA \quad (3.8)$$

- $EA\varepsilon_c^2$: Energy from axial extension/compression
- $EI_1\kappa_1^2$: Energy from lateral bending (bending in the plane of the wheel)
- $EI_2\kappa_2^2$: Energy from radial bending (bending out of the plane of the wheel)
- $GJ\kappa_3^2$: Energy from uniform (Saint-Venant) torsion
- $EI_{\omega}(\kappa_3')^2$: Energy from non-uniform torsion (warping)

Moving on to the spokes, each spoke contributes a tension-force directed along the spoke axis:

$$f = Tn \quad (3.9)$$

where T is the spoke tension (a positive scalar for tension) and n is the unit vector pointing from the spoke nipple (at the rim) to the hub eyelet. By Newton's third law, the rim exerts an equal and opposite force $-Tn$ on the spoke. Additionally we need to take into account the geometric- and material stiffness of each spoke, where the material stiffness of the spoke arises from elastic elongation of the wire:

$$K_s = \frac{E_s A_s}{l_s} \quad (3.10)$$

where $A_s = \pi d^2/4$ is the cross-sectional area. As mentioned above, a prestensioned spoke also possesses geometric stiffness in directions transverse to its axis. This effect, familiar from the mechanics of tensioned strings and membranes, arises from the change in force direction when the spoke rotates. Consider a spoke under pretension T^p that undergoes a small transverse displacement $\delta u_{\perp}$. The spoke rotates through an angle $\delta u_{\perp}/l_s$, causing the tension force to acquire a component $T^p(\delta u_{\perp}/l_s)$ perpendicular to the original spoke direction. This component acts as a restoring force with effective stiffness:

$$k_{geom} = \frac{T^p}{l_s} \quad (3.11)$$

The total force-displacement relationship for a spoke can then be written in matrix form. Let δu_n be the displacement of the spoke nipple relative to the hub. The incremental force on the spoke is:

$$\delta f = k_f \delta u_n \quad (3.12)$$

with the force-stiffness matrix being:

$$k_f = K_s(n \otimes n) + \frac{T^p}{l_s}(I - n \otimes n) \quad (3.13)$$

where I is the 3×3 identity matrix and $n \otimes n$ is the outer product of the spoke direction vector with itself. The first term represents material stiffness (acts only along n), while the second term represents geometric stiffness (acts in the plane perpendicular to n).

For coupling with the rim displacement, including the contribution in rotation, we have:

$$u_n = [u, v, \omega]^T + \phi e_3 \times b_s \quad (3.14)$$

For a spoke offset laterally by b_1 and radially by b_2 , the cross product gives:

$$\phi e_3 \times b_s = \phi[-b_2, b_1, 0]^T \quad (3.15)$$

need to write less and connect better to next equation (Spoke strain energy):

$$U_{spokes} = \sum_i [f_i^p \cdot \delta u_{n,i} + \frac{1}{2} d(\theta_i)^T k_i d(\theta_i)] \quad (3.16)$$

System Solution

Having established the energy expressions for the rim and the spoke system, we can now write down the total potential energy as:

$$\Pi = U_{rim} + U_{spokes} - W_{ext} \quad (3.17)$$

Minimizing this expression with respect to the displacement field $d(\theta)$ yields the equilibrium configuration. This is done with a modal decomposition method, based on the Rayleigh-Ritz approximation, reduces the problem to a finite-dimensional matrix equation by approximating $d(\theta)$ with a finite Fourier series. Due to the rim being a closed loop and therefore periodic by nature, we can write each displacement component as:

$$u(\theta) = u_0 + \sum_{n=1}^N [u_n^c \cos(n\theta) + u_n^s \sin(n\theta)] \quad (3.18)$$

$$v(\theta) = v_0 + \sum_{n=1}^N [v_n^c \cos(n\theta) + v_n^s \sin(n\theta)] \quad (3.19)$$

$$u(\theta) = \omega_0 + \sum_{n=1}^N [\omega_n^c \cos(n\theta) + \omega_n^s \sin(n\theta)] \quad (3.20)$$

$$u(\theta) = \phi_0 + \sum_{n=1}^N [\phi_n^c \cos(n\theta) + \phi_n^s \sin(n\theta)] \quad (3.21)$$

Here N is the maximum mode number included in the approximation. As $N \rightarrow \infty$, the series converges to the exact displacement field.

We collect all the Fourier coefficients into a mode-coefficient vector d_m :

$$d_m = [u_0, v_0, \omega_0, \phi_0, u_1^c, u_1^s, v_1^c, v_1^s, \omega_1^c, \omega_1^s, \pi_1^c, \phi_1^s, \dots, \pi_N^c, \phi_N^s]^T \quad (3.22)$$

which now stands in for our displacement field as the fundamental unknown. To express the relationship between d_m and $d(\theta)$, we define the mode matrix $B(\theta)$:

$$d(\theta) = B(\theta)d_m \quad (3.23)$$

NOTE: $T = (2\pi/n_s)\bar{T}$

$$B(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 & c\theta & s\theta & 0 & 0 & 0 & 0 & 0 & 0 & 2c\theta & 2s\theta & \dots \\ 0 & 1 & 0 & 0 & 0 & 0 & c\theta & s\theta & 0 & 0 & 0 & 0 & 0 & 0 & \dots \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & c\theta & s\theta & 0 & 0 & 0 & 0 & \dots \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & c\theta & s\theta & 0 & 0 & \dots \end{bmatrix}$$

where $c\theta = \cos\theta$, $s\theta = \sin\theta$, etc. From the series approximation we get the quadratic:

$$\Pi_{rim} = \frac{1}{2} d_m^T K_{rim} d_m \quad (3.24)$$

Where K_{rim} is the mode-stiffness matrix:

$$K_{rim} = \begin{bmatrix} K_0^{rim} & 0 & 0 & 0 \\ 0 & K_1^{rim} & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & K_N^{rim} \end{bmatrix}$$

The zero-mode matrix is

$$K_0^{rim} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 2\pi \frac{EA}{R} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2\pi \frac{EI_2}{R} + 2\pi R \bar{T} y_0 \end{bmatrix}$$

and the subsequent matrices have the form

$$K_{n \geq 1}^{rim} = \begin{bmatrix} k_{uu} & 0 & 0 & 0 & 0 & 0 & -k_{u\phi} & 0 \\ 0 & k_{uu} & 0 & 0 & 0 & 0 & 0 & -k_{u\phi} \\ 0 & 0 & k_{vv} & 0 & 0 & -k_{v\omega} & 0 & 0 \\ 0 & 0 & 0 & k_{vv} & k_{v\omega} & 0 & 0 & 0 \\ 0 & 0 & -k_{v\omega} & 0 & 0 & k_{\omega\omega} & 0 & 0 \\ -k_{u\phi} & 0 & 0 & 0 & 0 & 0 & k_{\phi\phi} & 0 \\ 0 & -k_{u\phi} & 0 & 0 & 0 & 0 & 0 & k_{\phi\phi} \end{bmatrix} \quad (3.25)$$

where

$$\begin{aligned} k_{uu} &= \frac{\pi EI_2}{R^3} n^4 + \frac{\pi EI\omega}{R^5} n^4 + \frac{\pi GJ}{R^3} n^2 - \pi n^2 \bar{T} \left(1 + \frac{r_0^2}{R^2}\right) \\ k_{vv} &= \frac{\pi EI_2}{R^3} n^4 + \frac{\pi EA}{R} \left(1 + \frac{y_0}{R} n^2\right)^2 \\ k_{\omega\omega} &= \frac{\pi EI_2}{R^3} n^2 + \frac{\pi EAn^2}{R} \left(1 + \frac{y_0}{R}\right)^2 \\ k_{v\omega} &= \frac{\pi EI_2}{R^3} n^3 + \frac{\pi EAn}{R} \left(1 + \frac{y_0}{R} (1 + n^2) + \frac{y_0^2}{R^2} n^2\right) \\ k_{\phi\phi} &= \frac{\pi EI_2}{R} + \frac{\pi EI\omega}{R^3} n^4 + \frac{\pi GJ}{R} n^2 + \pi R^2 \bar{T} \left(\frac{y_0}{R} - \frac{r_0^2}{R^2} n^2\right) \\ k_{u\phi} &= \frac{\pi EI_2}{R^2} n^2 + \frac{\pi EI\omega}{R^4} n^4 + \frac{\pi GJ}{R^2} n^2 + \pi n^2 R \bar{T} \left(\frac{y_0}{R} - \frac{r_0^2}{R^2} n^2\right) \end{aligned}$$

strain energy spokes:

$$U_{spokes} = \frac{1}{2} \sum_{i=1}^{n_s} d_i^T k_i d_i \quad (3.26)$$

$$= \frac{1}{2} \sum_{i=1}^{n_s} (B_i^T k_i B_i) d_m \quad (3.27)$$

$$= \frac{1}{2} d_m^T \left(\sum_{i=1}^{n_s} B_i^T k_i B_i \right) d_m \quad (3.28)$$

$$= \frac{1}{2} d_m^T K_{spk} d_m \quad (3.29)$$

combined:

$$\Pi = \frac{1}{2} d_m^T (K_{rim} + K_{spk}) d_m - F_{ext} d_m \quad (3.30)$$

minimizing:

$$(K_{rim} + K_{spk}) d_m = F_{ext} \quad (3.31)$$

truing case:

$$(K_{rim} + K_{spk})d_m = 0 \quad (3.32)$$

On Wheel Parameters

The properties of the wheel simulation are characterized by the parameters shown in Table 3.1.

Table 3.1: Wheel simulation parameters

Parameter	Value	Description
n_spokes	36	Number of spokes
max_tension_threshold	1200 N	Maximum allowable spoke tension
max_tension_penalty	true	Enable penalty for exceeding max tension
hub_width	50 mm	Hub width (distance between flanges)
hub_diameter	40 mm	Hub flange diameter
rim_radius	300 mm	Rim radius (at shear center)
rim_area	100 mm ²	Rim cross-sectional area
rim_I_lat	1500 mm ⁴	Rim lateral moment of inertia (I_2)
rim_I_rad	3000 mm ⁴	Rim radial moment of inertia (I_1)
rim_J_tor	500 mm ⁴	Rim torsion constant (J)
rim_young_mod	69 GPa	Rim Young's modulus (aluminum)
rim_shear_mod	26 GPa	Rim shear modulus
rim_I_warp	0.0 mm ⁶	Rim warping constant (I_w)
spokes_crossings	3	Number of spoke crossings (lacing pattern)
spokes_diameter	2.0 mm	Spoke wire diameter
spokes_young_mod	210 GPa	Spoke Young's modulus (steel)
number_modes	40	Number of Fourier modes (N)
init_tension	800 N	Initial spoke pretension (T_p)

3.1.2 On State-Space Formulations

The complete state is defined by the displacement field and spoke tensions in equilibrium as explained in the previous chapter (link or explicitly name). For representing the state we can choose an arbitrary number of points around the rim in terms of the rotation angle θ . We make this choice to be 360 points, equidistant around the circle, i.e.: $s = [\theta_1, \theta_1, \dots, \theta_{360}]$ For each of these points, we can simulate the displacement triad, giving us the local lateral, tangential and radial displacement. As it is not realistic to discern the tangential displacement by eye when truing a wheel, or even measure it through sensors, it is omitted from the state. The spoke tensions in equilibrium are calculated by the simulation in units of Newtons. To reduce the variation in absolute values in the state (displacement vectors on the order of a few millimeters vs. spoke tensions on the order of hundreds of Newtons), the tensions are normalized by dividing

through the initial tension. Since the displacement around the wheel is periodic by nature, we can also express this displacement in terms of its Fourier coefficients. For the lateral and radial displacement fields $u_{\text{lat}}(\theta)$ and $u_{\text{rad}}(\theta)$, we compute the discrete Fourier transform coefficients up to N harmonics:

$$c_k = \frac{1}{M} \sum_{j=0}^{M-1} u(\theta_j) e^{-2\pi i k j / M}, \quad k = 0, 1, \dots, N \quad (3.33)$$

where $M = 360$ is the number of sample points, $\theta_j = 2\pi j / M$ are the equidistant angular positions, and $u(\theta_j)$ represents either the lateral or radial displacement at position j . The Fourier representation consists of the real and imaginary parts of these coefficients for both displacement components, yielding a more compact state representation of $4(N + 1)$ features. We choose $N = 30$ by the elbow-graph heuristic on the distance of the original displacement points to the reconstructed points from the Fourier coefficients with some margin of safety. To explore how much of the full state is required for the agent to learn and if we could get away without measuring the spokes in a potential physical implementation, we combine these parts into the following state-space options:

- only displacement points
- displacement points and normalized spoke tensions
- Fourier coefficients
- Fourier coefficients and normalized spoke tensions

3.1.3 On the Starting State

On reset of the environment, i.e., start of a new episode, we need to construct an out-of-true wheel. We do this by starting with a true wheel with completely uniform tensions and then bringing it out of true by choosing n_{choice} spokes and turning them some random uniformly sampled value from $[-\max_{\text{turn}}, \max_{\text{turn}}]$. We choose somewhat arbitrarily $n_{\text{choice}} = 5$ and $\max_{\text{turn}} = 1$.

3.1.4 On Action-Space Formulations

Since DQN variants only work with discrete and TDMPC2 only with continuous action spaces, we need at least two formulations. Starting with the discrete formulation, we let the agent choose from $[2 \cdot n_{\text{spokes}}]$ actions corresponding to a 0.1 turn in $+/-$ direction of the corresponding spoke. We can transform this directly into a 2D continuous action-space by letting the first variable control which spoke should be adjusted and letting the second control the adjustment size. Additionally, we can also formulate a

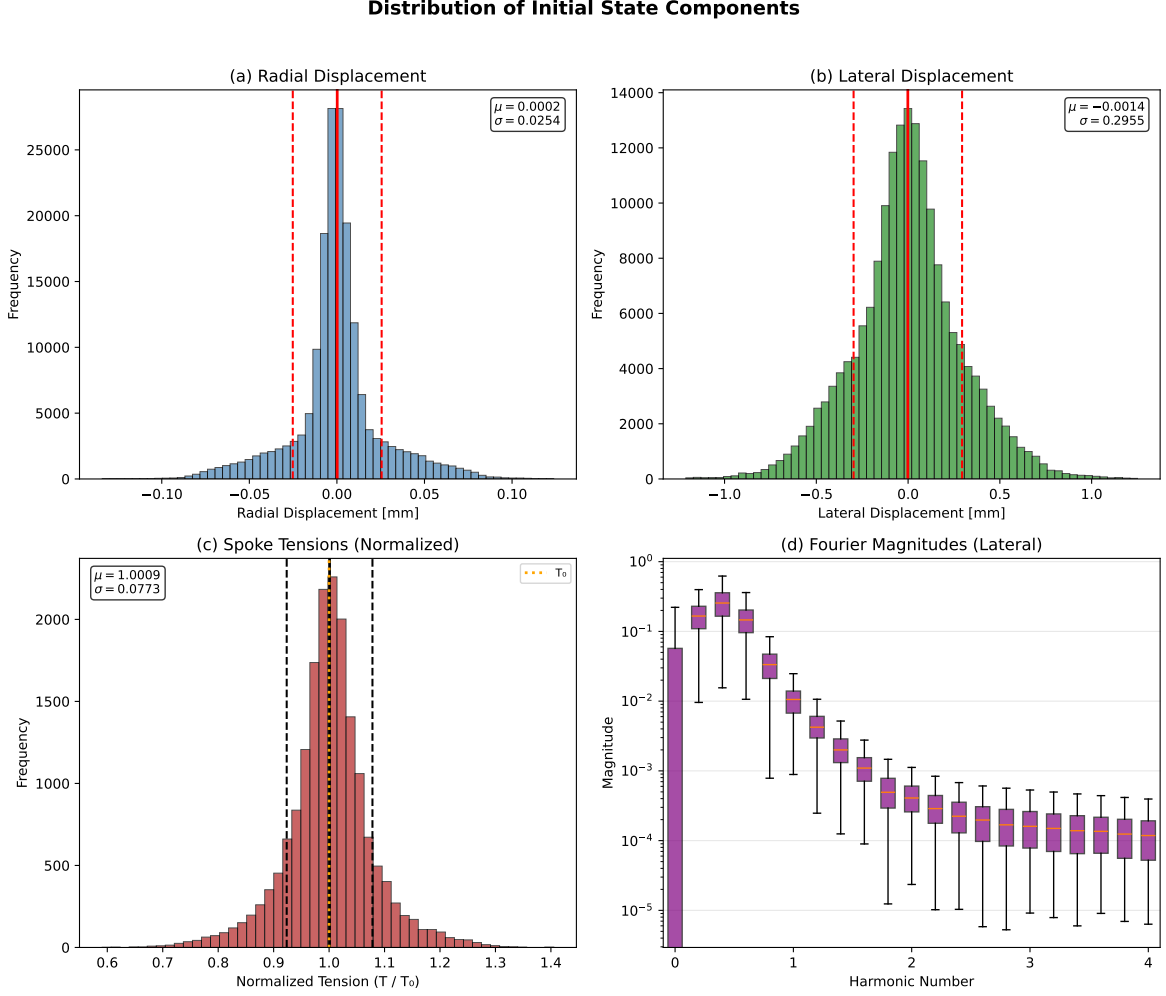


Figure 3.1: Distribution of initial state components across 500 sampled wheel configurations. (a) Radial displacements are approximately an order of magnitude smaller than (b) lateral displacements. (c) Spoke tensions are normalized by the initial tension $T_0 = 800$ N, centering the distribution around 1.0. (d) The Fourier magnitude spectrum shows that lower harmonics dominate the initial disturbances. Red lines indicate mean and ± 1 standard deviation.

continuous action by letting the agent adjust all the spokes at once with a vector of length n_{spokes} , giving us the following three action spaces $\mathcal{A}$:

- **Continuous spoke selection:** two-dimensional continuous space

$$\mathcal{A}_{\text{continuous}} = [0, n_{\text{spokes}} - 1] \times [-1, 1] \subset \mathbb{R}^2 \quad (3.34)$$

where n_s is the number of spokes. An action $a = (s, \Delta t) \in \mathcal{A}_{\text{continuous}}$ specifies a continuous spoke index s and a tension adjustment Δt .

- **Discrete spoke selection:**

$$\mathcal{A}_{\text{discrete}} = \{0, 1, 2, \dots, 71\} \quad (3.35)$$

representing 72 distinct action choices, where each action corresponds to a specific spoke-tension adjustment pair.

- **All spokes adjustment:** n_{spokes} -dimensional continuous space

$$\mathcal{A}_{\text{all_spokes}} = [-1, 1]^{n_{\text{spokes}}} \subset \mathbb{R}^{n_s} \quad (3.36)$$

where an action $a = (\Delta t_1, \Delta t_2, \dots, \Delta t_{n_s}) \in \mathcal{A}_{\text{all_spokes}}$ specifies simultaneous tension adjustments for all spokes.

For the continuous action space formulation we make a special choice for the PPO-Algorithm: Treating both dimensions with a shared Gaussian distribution conflates two fundamentally different quantities, a discrete choice among the spokes and a bounded continuous adjustment—making it difficult to balance exploration appropriately across both. We therefore opt for a two-headed network design to reflect this discrete-continuous dichotomy.

Algorithm–Action Space Compatibility

Not all algorithms can be evaluated on all action space formulations due to inherent design constraints. Table 3.2 summarizes the valid combinations.

Table 3.2: Algorithm–action space compatibility matrix. Checkmarks indicate valid combinations tested in this work.

Algorithm	Discrete	Continuous	All-Spokes
Rainbow DQN	✓	—	—
PPO	✓	✓	✓
TD-MPC2	—	✓	✓

The constraints arise from fundamental algorithmic requirements:

- **Rainbow DQN:** Requires enumeration of Q-values for all actions, restricting it to discrete formulations.
- **TD-MPC2:** Performs gradient-based trajectory optimization, requiring differentiable (continuous) action spaces.
- **PPO:** Accommodates both discrete (categorical policy) and continuous (Gaussian policy) actions through appropriate policy parameterization.

This compatibility structure has implications for cross-algorithm comparisons: direct comparison is only possible where action spaces overlap. PPO serves as a bridge, being the only algorithm evaluated on all three action formulations.

3.1.5 On Reward Functions

The formulation of the reward is notoriously important for successful reinforcement learning.

Generally, we can construct a scalar reward from a given wheel-state by either looking at the remaining spoke turns from its initial true state or by measuring the total displacement by summing over the displacement vectors weighted by $d\theta$:

$$\|s_t\| \approx \sqrt{\sum_{k=1}^{N-1} \frac{\Delta\theta_k}{2} [\|u(\theta_k)\|_2^2 + \|u(\theta_{k+1})\|_2^2]} \quad (3.37)$$

where $\|u(\theta)\|_2^2 = \sum_{d=1}^D u_d^2(\theta)$ is the squared Euclidean norm of the displacement vector at angle θ , and define the step-improvement I_{step} as $\|s_{t-1}\| - \|s_t\|$.

- **Raw reward:** Directly penalizes the remaining displacement in the rim

$$r_t^{\text{raw}} = -\|s_t\| \quad (3.38)$$

- **Normalized reward:** Provides binary feedback based on step-wise improvement

$$r_t^{\text{normalized}} = \begin{cases} 0 & \text{if } I_{\text{step}} > 0 \\ -1 & \text{if } I_{\text{step}} \leq 0 \end{cases} \quad (3.39)$$

- **Spokes reward:** Provides binary feedback based on spoke adjustment magnitude

$$r_t^{\text{spokes}} = \begin{cases} 0 & \text{if } |\Delta t_i^{(t)}| \leq |\Delta t_i^{(t-1)}| \text{ for all } i \\ -1 & \text{otherwise} \end{cases} \quad (3.40)$$

where $\Delta t_i^{(t)}$ denotes the turns applied to spoke i at time step t .

Additionally, we introduce two stopping conditions to reward the agent for producing a true wheel or punish for exceeding a tension threshold. Some research into actual manual wheel truing shows that the "trueness" is often evaluated only by the rim displacement with less concern for the uniformity of spoke tensions. The state of a true wheel is often seen as dependent on the material quality of the wheel. In this context, we often see discussion around the concepts of "decent," "good," and "golden" wheels, where these generally have the following specifications:

- A decent wheel refers to a common quality wheel which is considered reasonably true with a maximum rim displacement of 1–2 mm and a relative tension deviation of 15–20%

- A good wheel refers to a good quality wheel with a maximum rim displacement of 0.5–1 mm and a relative tension deviation of 5–10%
- A golden wheel refers to an excellent quality wheel with a maximum rim displacement of less than 0.5 mm and a relative tension deviation of under 5%

Since in simulation we can assume no errors or inherent tension in the material of the rim, we can orient ourselves close to the golden standard. From this we introduce the success condition as:

$$|u_d(\theta_i)| < 0.3 \text{ mm} \quad \forall d \in \{1, \dots, D\}, \forall i \in \{1, \dots, N\} \quad (3.41)$$

$$700 \text{ N} \leq T_j \leq 900 \text{ N} \quad \forall j \in \{1, \dots, n_s\} \quad (3.42)$$

$$\max_j T_j - \min_j T_j \leq 0.1 \cdot \bar{T} \quad (3.43)$$

where $u_d(\theta_i)$ represents the displacement in dimension d at discrete angle θ_i , T_j is the tension in spoke j , and $\bar{T} = \frac{1}{n_s} \sum_{j=1}^{n_s} T_j$ is the mean spoke tension. When these conditions are met, the episode terminates with a bonus reward of $r_t = 20$.

For the penalty condition we consider the fact that most wheel manufacturers give a maximum tension of 120 kg force on a spoke. We consider the episode as failed when a single spoke exceeds this threshold and we punish the agent with a reward of $r_t = -20$. To avoid filling the replay buffer with exceptionally bad states at the beginning, we also terminate the episode after exceeding this threshold. So our second stopping condition can be formulated as:

$$\text{if } \exists j \in \{1, \dots, n_s\} : T_j > T_{\max} \quad \text{then} \quad r_t \leftarrow r_t - 20, \text{ truncated} = \text{True} \quad (3.44)$$

where $T_{\max}$ is the maximum tension threshold.

Important: *This truncation condition has shown itself to be exploitable in the continuous action spaces and was changed for PPO and TD-MPC2 to:*

$$\text{if } \exists j \in \{1, \dots, n_s\} : T_j > T_{\max} \quad \text{then} \quad r_t \leftarrow r_t - 2, \text{ truncated} = \text{False} \quad (3.45)$$

3.2 Algorithm Configurations

This section describes the specific configurations of each reinforcement learning algorithm used in the experiments. While the algorithmic foundations were introduced in

Section ??, the focus here is on implementation choices, network architectures, and hyperparameter settings.

3.2.1 Rainbow DQN

Implementation Basis The Rainbow DQN implementation is adapted from the open-source PyTorch implementation by Kaixhin¹. The primary modification replaces the original Categorical DQN (C51) distributional component with Implicit Quantile Networks (IQN) [3], which avoids the need to predefine a fixed set of atoms for the value distribution and instead learns the full quantile function by conditioning on sampled quantile fractions $\tau \in [0, 1]$.

Rainbow Components The implementation incorporates the following components from the Rainbow framework:

- **Double DQN** [4]: action selection uses the online network while evaluation uses the target network which is updated every 8000 steps, reducing overestimation bias.
- **Dueling architecture** [5]: the network separately estimates state-value $V(s)$ and advantage $A(s, a)$ streams, combining them as $Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$.
- **Distributional RL**: Implicit Quantile Networks (IQN) [3] to avoid having to choose a viable base for the distribution, rather than learning fixed quantiles, IQN learns the full quantile function by conditioning on sampled quantile fractions $\tau \in [0, 1]$.
- **Prioritized Experience Replay** [6]: transitions are sampled proportionally to their temporal-difference error with priority exponent $\alpha = 0.5$.
- **Noisy networks** [7]: all linear layers use factorized Gaussian noise with initial standard deviation $\sigma_0 = 0.1$, replacing ε -greedy exploration.
- **Multi-step returns**: 3-step returns are used, providing a balance between bias and variance in the bootstrap target.

Network Architecture The Q-network follows a dueling architecture with IQN-style quantile conditioning. The architecture processes states through parallel value and advantage streams:

¹<https://github.com/Kaixhin/Rainbow>

- **Input layer:** State dimension varies by representation (see Table ??). With `history_length = 1`, the input dimension equals the raw state dimension.
- **Value stream:** One noisy linear layer mapping input to 500 hidden units with ReLU activation, followed by quantile-conditioned processing and a final noisy linear layer outputting a scalar value.
- **Advantage stream:** Parallel structure to the value stream, with the final layer outputting 72 advantage values (one per action).
- **Quantile embedding:** A cosine embedding network with embedding dimension 64, mapping sampled quantile fractions τ to hidden representations via:

$$\phi(\tau) = \text{ReLU} \left(\sum_{i=0}^{63} \cos(\pi i \tau) \cdot \mathbf{w}_i + b \right) \quad (3.46)$$

The quantile embeddings are combined with state features through element-wise multiplication.

The complete forward pass samples $N = 32$ quantile fractions uniformly from $[0, 1]$, producing Q-value distributions represented as $(|\mathcal{A}| \times N)$ quantile values per state.

Noisy Layer Implementation The noisy linear layers implement factorized Gaussian noise [7]. For a layer with p inputs and q outputs, the weight noise is factorized as:

$$\varepsilon_{i,j}^w = f(\varepsilon_i) f(\varepsilon_j), \quad \text{where } f(x) = \text{sgn}(x) \sqrt{|x|} \quad (3.47)$$

This reduces the number of random variables from $pq + q$ to $p + q$ while maintaining sufficient exploration. During evaluation, noise is disabled and only the mean weights μ^w are used.

Hyperparameters Table 3.3 summarizes the hyperparameters used for Rainbow DQN training.

Loss Function The network is trained using the quantile Huber loss. For a transition (s, a, r, s') with sampled quantile fractions $\tau_1, \dots, \tau_N$ for the online network and $\hat{\tau}_1, \dots, \hat{\tau}_{N'}$ for the target, the loss is:

$$\mathcal{L} = \frac{1}{N \cdot N'} \sum_{i=1}^N \sum_{j=1}^{N'} \rho_{\tau_i}^{\kappa}(\delta_{ij}) \quad (3.48)$$

Table 3.3: Rainbow DQN hyperparameters

Parameter	Value
<i>Learning</i>	
Learning rate	5.25×10^{-5}
Adam ε	1.5×10^{-4}
Discount factor γ	0.99
Gradient norm clipping	10.0
<i>Replay Buffer</i>	
Replay buffer capacity	1×10^6
Batch size	64
Learning starts	20,000 steps
Replay frequency	Every 4 steps
Priority exponent α	0.5
Priority weight β	0.4
<i>Target Network</i>	
Target network update	Every 8,000 steps
<i>Architecture</i>	
Hidden layer size	500
Noisy layer σ_0	0.1
Multi-step returns n	3
<i>IQN</i>	
Training quantiles N	32
Evaluation quantiles K	32
Quantile embedding dimension	64
Huber loss threshold κ	1.0

where $\delta_{ij} = r + \gamma Z_{\hat{\tau}_j}(s', a^*) - Z_{\tau_i}(s, a)$ is the temporal-difference error, $a^* = \arg \max_a \mathbb{E}[Z(s', a)]$ follows Double DQN, and ρ_τ^κ is the quantile Huber loss:

$$\rho_\tau^\kappa(\delta) = |\tau - \mathbf{1}_{\delta < 0}| \cdot \mathcal{L}_\kappa(\delta), \quad \mathcal{L}_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{if } |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{1}{2}\kappa) & \text{otherwise} \end{cases} \quad (3.49)$$

3.2.2 PPO

Implementation Basis The PPO implementation is adapted from the open-source PyTorch implementation by Saqib². Several modifications were made to support the wheel truing task:

- **Hybrid actor architecture:** A dual-headed actor network (`HybridPPOAgent`) was implemented to support the continuous action space formulation, combining discrete spoke selection (categorical distribution) with continuous turn magnitude (Gaussian distribution).
- **Vectorized rollout collection:** The training loop was extended to support parallel environment execution using subprocess-based vectorization, improving sample throughput and GPU utilization.

Network Architecture The policy uses an actor-critic architecture with a shared feature extractor:

- **Shared feature extractor:** Two fully-connected layers with 500 units each and Tanh activations, processing the input state.
- **Actor head:** Action-space-dependent output layer (see below).
- **Critic head:** Single linear layer outputting a scalar value estimate.

The actor head is parameterized differently for each action space formulation:

- **Discrete (72 actions):** Linear layer followed by softmax, outputting a categorical distribution over 72 actions (36 spokes $\times$ 2 directions).
- **Continuous/Hybrid (2D):** A hybrid architecture using a separate `HybridPPOAgent` that combines discrete spoke selection (categorical distribution over 36 spokes) with a continuous turn magnitude (Gaussian distribution). This formulation allows the agent to select which spoke to adjust and by how much in a single action.

²<https://github.com/saqib1707/RL-PPO-PyTorch>

- **All-spokes (36D)**: Linear layer outputting 36 means for independent Gaussian distributions, one per spoke. The policy outputs simultaneous adjustments for all spokes.

Action Distribution For continuous action dimensions, the policy uses a multivariate Gaussian distribution with diagonal covariance:

$$\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma^2)) \quad (3.50)$$

where $\mu_{\theta}(s)$ is the network output and σ^2 is a *learned parameter* initialized to $\sigma_0^2 = 0.1^2$. The variance is implemented as a trainable `nn.Parameter`, allowing the policy to adapt its exploration over training.

Vectorized Training To improve sample throughput, training employs $n_{\text{envs}} = 8$ parallel environment instances using subprocess-based vectorization. Each policy update uses

$$T_{\text{update}} = T_{\text{interval}} = 1000 \text{ steps} \quad (3.51)$$

collected across all environments, corresponding to 125 steps per environment.

Generalized Advantage Estimation Advantages are computed using Generalized Advantage Estimation (GAE) [8]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (3.52)$$

with discount factor $\gamma = 0.99$ and GAE parameter $\lambda = 0.95$. Advantages are normalized to zero mean and unit variance before each update.

Optimization The policy is updated using the clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (3.53)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\varepsilon = 0.2$ is the clip parameter.

The total loss combines policy, value, and entropy terms:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} + c_v \mathcal{L}^{\text{VF}} - c_e H[\pi_{\theta}] \quad (3.54)$$

where $\mathcal{L}^{\text{VF}} = (V_{\theta}(s_t) - R_t)^2$ is the value function loss, $H[\pi_{\theta}]$ is the policy entropy, $c_v = 0.5$ is the value loss coefficient, and $c_e = 0.01$ is the entropy coefficient.

A single Adam optimizer is used with parameter-group-specific learning rates:

- Shared feature extractor: default learning rate

- Actor head: 3×10^{-4}
- Critic head: 1×10^{-3}

Hyperparameters Table 3.4 summarizes the hyperparameters used for PPO training.

Table 3.4: PPO hyperparameters

Parameter	Value
<i>Learning</i>	
Actor learning rate	3×10^{-4}
Critic learning rate	1×10^{-3}
Discount factor γ	0.99
GAE λ	0.95
Max gradient norm	0.5
<i>PPO-Specific</i>	
Clip ratio ε	0.2
Entropy coefficient c_e	0.01
Value loss coefficient c_v	0.5
<i>Batching</i>	
Number of parallel environments	8
Update interval (total steps)	1000
Batch size	256
Epochs per update	10
<i>Architecture</i>	
Hidden layer size	500
Number of hidden layers	2
Activation function	Tanh
Action std init σ_0 (continuous)	0.1

Action Spaces Tested PPO is the only algorithm evaluated on all three action space formulations, enabled by a factory pattern that instantiates the appropriate agent variant based on the `action_space_type` configuration parameter:

- **Discrete:** Standard `PP0Agent` with categorical policy over 72 actions
- **Continuous/Hybrid:** `HybridPP0Agent` combining discrete spoke selection with continuous turn magnitude
- **All-spokes:** Standard `PP0Agent` with 36-dimensional Gaussian policy

This flexibility makes PPO the only algorithm that enables direct comparison across all action formulations.

3.2.3 TD-MPC2

Implementation Basis The TD-MPC2 implementation is based on the JAX/Flax re-implementation by Flandermeyer³, which itself follows the original TD-MPC2 algorithm [9]. The implementation was left largely unchanged, with only minor modifications to support configurable network sizes for the encoder and world model components.

Architecture Overview TD-MPC2 jointly learns five neural network components that together form a world model for latent-space planning:

- **Encoder** h_θ : Maps observations to latent states $z = h_\theta(s)$
- **Dynamics model** d_θ : Predicts next latent state $z' = d_\theta(z, a)$
- **Reward model** r_θ : Predicts reward $\hat{r} = r_\theta(z, a)$
- **Policy model** π_θ : Outputs action distribution $\pi_\theta(a|z)$
- **Value model** Q_θ : Estimates action-value $Q_\theta(z, a)$

All networks use normalized linear layers (**NormedLinear**) with layer normalization and Mish activation [10]:

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (3.55)$$

The latent space uses SimNorm [9] as the final activation, which projects representations onto a simplex:

$$\text{SimNorm}(x) = \text{softmax}(\text{reshape}(x, [L, V])) \quad (3.56)$$

where $L = \text{latent_dim}/V$ and $V = 8$ is the simplex dimension.

Network Architectures Table 3.5 details the architecture of each component.

Table 3.5: TD-MPC2 network architectures. All hidden layers use Mish activation with layer normalization.

Component	Architecture	Output
Encoder	$s \rightarrow 256 \rightarrow 512$ (SimNorm)	Latent state z
Dynamics	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 512$ (SimNorm)	Next latent z'
Reward	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 101$	Reward bins
Policy	$z \rightarrow 512 \rightarrow 512 \rightarrow 2 \cdot A $	Mean, log-std
Value	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 101$	Q-value bins

³<https://github.com/ShaneFlandermeyer/tdmpc2-jax>

Distributional Value and Reward Prediction Following TD-MPC2, both reward and value predictions use a distributional representation with two-hot encoding. Values are discretized into 101 bins spanning $[-10, 10]$ in symlog space:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(1 + |x|) \quad (3.57)$$

The two-hot encoding provides soft targets for regression, and predictions are recovered via:

$$\hat{v} = \text{symexp} \left(\sum_{i=1}^{101} \text{softmax}(\text{logits})_i \cdot b_i \right) \quad (3.58)$$

where b_i are the bin centers and $\text{symexp}(x) = \text{sign}(x)(e^{|x|} - 1)$.

Value Ensemble The value function uses an ensemble of $N_Q = 5$ networks with dropout ($p = 0.01$) for uncertainty estimation. During policy optimization, the mean across ensemble members is used; during TD-target computation, two randomly sampled members are used with minimum aggregation to reduce overestimation.

Planning with MPPI At inference time, actions are selected via Model Predictive Path Integral (MPPI) control in latent space. The planning procedure:

1. Encode current observation: $z_0 = h_\theta(s)$
2. Initialize action sequences: 24 from policy rollouts, 488 from Gaussian noise
3. For each MPPI iteration (6 total):
 - (a) Evaluate all trajectories using learned dynamics and rewards
 - (b) Select top 64 elites based on estimated returns
 - (c) Update mean and std via softmax-weighted aggregation
4. Sample final action from elite distribution using Gumbel-softmax

The return estimate for a trajectory combines imagined rewards with a terminal value:

$$G = \sum_{t=0}^{H-1} \gamma^t \hat{r}_t + \gamma^H Q_\theta(z_H, \pi_\theta(z_H)) \quad (3.59)$$

Training Objective The world model is trained to minimize a weighted combination of losses:

$$\mathcal{L} = \lambda_c \mathcal{L}_{\text{consistency}} + \lambda_r \mathcal{L}_{\text{reward}} + \lambda_v \mathcal{L}_{\text{value}} \quad (3.60)$$

where:

- $\mathcal{L}_{\text{consistency}}$: MSE between predicted and encoded next latent states

- $\mathcal{L}_{\text{reward}}$: Soft cross-entropy for reward prediction
- $\mathcal{L}_{\text{value}}$: Soft cross-entropy for Q-value prediction with TD targets

The policy is trained separately to maximize Q-values with entropy regularization:

$$\mathcal{L}_{\pi} = \mathbb{E}_{z,a \sim \pi} [\alpha \log \pi(a|z) - Q(z, a)] \quad (3.61)$$

Hyperparameters Tables 3.6 and 3.7 summarize the hyperparameters.

Table 3.6: TD-MPC2 planning hyperparameters

Parameter	Value
Planning horizon H	3
MPPI iterations	6
Population size	512
Policy prior samples	24
Number of elites	64
Minimum plan std	0.05
Maximum plan std	2.0
Temperature	1.0

Table 3.7: TD-MPC2 training hyperparameters

Parameter	Value
<i>Architecture</i>	
Latent dimension	512
MLP hidden dimension	512
Encoder hidden dimension	256
Number of encoder layers	2
SimNorm dimension V	8
Number of value networks N_Q	5
Value dropout	0.01
Number of bins	101
Symlog range	$[-10, 10]$
<i>Optimization</i>	
Learning rate (all networks)	3×10^{-4}
Batch size	256
Discount factor γ	0.99
Horizon decay ρ	0.5
Max gradient norm	20
Target network τ	0.01
<i>Loss Coefficients</i>	
Consistency λ_c	20
Reward λ_r	0.1
Value λ_v	0.1
Entropy α	10^{-4}
<i>Data Collection</i>	
Number of parallel environments	7
Replay buffer size	300,000
Seed steps	$\max(5 \cdot 250, 1000) \cdot 7 \cdot 1.0$
Update-to-data ratio	1.0

3.2.4 Common Training Configuration

To ensure comparability, all algorithms share the following training configuration:

Table 3.8: Shared training configuration across all algorithms

Parameter	Value
Total environment steps	1×10^6
Maximum episode length	41 steps
Training runs per configuration	1

Logging Training progress is logged using TensorBoard. The following metrics are recorded for every episode:

- Episodic return
- Final state norm (rim displacement)
- Maximum rim displacement component
- Absolute sum of spoke tension deviations from 800
- Maximum spoke tension deviation from 800
- Episode termination type (success / failure / timeout)

3.3 Experimental Design

This chapter describes the experimental methodology used to analyze the influence of state representation, action formulation, reward design, and optimization strategy on reinforcement learning performance in the wheel truing task. The experiments are structured as a controlled ablation study, followed by targeted hyperparameter optimization and optional investigations into temporal state augmentation.

The primary goal of the experimental design is not to obtain optimal absolute performance, but to systematically compare modeling and algorithmic choices in a physics-based reinforcement learning environment under a fixed computational budget.

3.3.1 Ablation Study Design

The ablation study forms the core of the experimental analysis. Its purpose is to identify which components of the reinforcement learning formulation are most influential for learning a successful wheel truing policy. To this end, multiple design dimensions are varied systematically while all other aspects of the environment and training procedure are held constant.

Ablation dimensions

The ablation study systematically varies the following components of the reinforcement learning setup:

- **State-space representation:** Rim displacement representations (point-based and Fourier-based), spoke-tension-only representations, and combined representations including both geometric and tension information.
- **Action-space formulation:** Discrete spoke adjustments, continuous single-spoke actions, and continuous multi-spoke adjustments.

- **Reward function design:** Reward formulations based on absolute rim displacement, step-wise improvement, and penalties related to spoke adjustment effort.
- **Reinforcement learning algorithm:** Value-based (DQN), policy-gradient (PPO), and model-based (TD-MPC2) approaches.

These dimensions were chosen to reflect both practical considerations in wheel truing (e.g., observability and control granularity) and common design choices in reinforcement learning. All experiments in the ablation study employ feedforward neural network policies without explicit recurrence. This restriction is intentional, as it allows the influence of state representation and reward design to be analyzed without the confounding effects of learned temporal memory. Temporal augmentation strategies are investigated separately in a later experimental phase.

Controlled comparison strategy

To isolate the influence of individual design choices, experiments are conducted under a controlled comparison protocol. Wherever possible, only a single ablation dimension is varied at a time, while all other aspects of the training setup are held fixed.

Specifically, episode length, termination conditions, wheel parameters, and training budgets are identical across all configurations. Each configuration is trained once under stochastic but fixed conditions, reflecting a single realization of the learning process. Due to computational constraints imposed by the available resources, stochastic variability across multiple training runs is not explicitly characterized. Instead, the experimental design prioritizes breadth of methodological comparison over statistical estimation of expected performance. Conclusions are therefore drawn based on relative performance trends and qualitative behavioral differences between configurations rather than precise quantitative comparisons.

3.3.2 Methodological Limitations

The primary limitations of the methodology include:

- Use of a single training run per configuration
- Uncontrolled stochasticity during training
- Quasi-static simulation assumptions
- Absence of sim-to-real validation

As a result, conclusions focus on methodological trends and comparative behavior rather than statistically significant performance estimates.

Chapter 4

Results and Analysis

4.1 Baseline Performance

- performance of the algorithms in phase 1
- what can we say about the influence of reward functions and
- state space
- action space
- cross factor analysis

4.1.1 distribution of metrics at reset

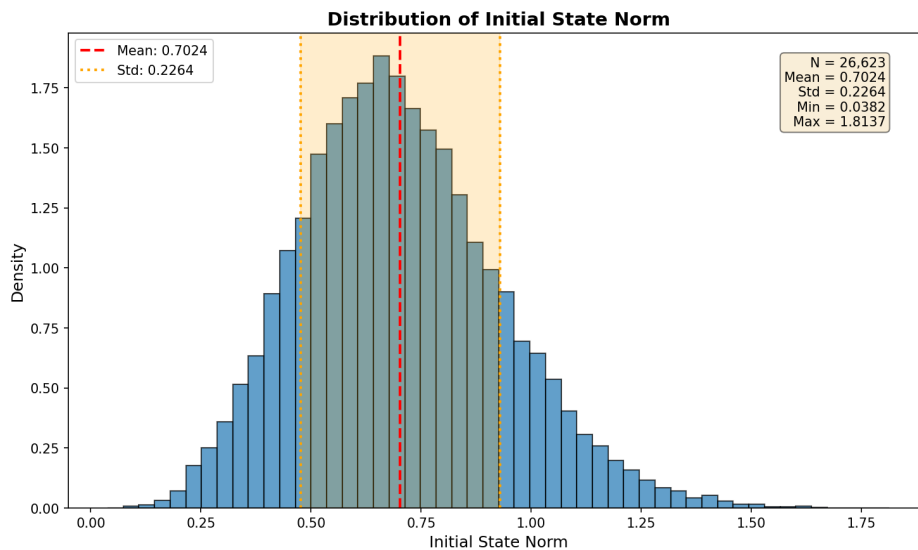


Figure 4.1: initial state norm distribution.

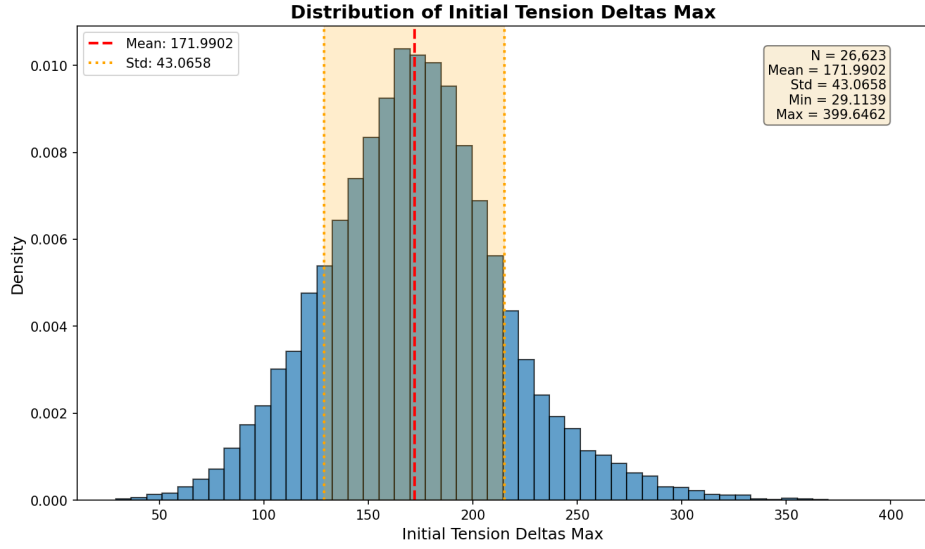


Figure 4.2: initial maximum tension deviation (from 800).

4.1.2 DQN

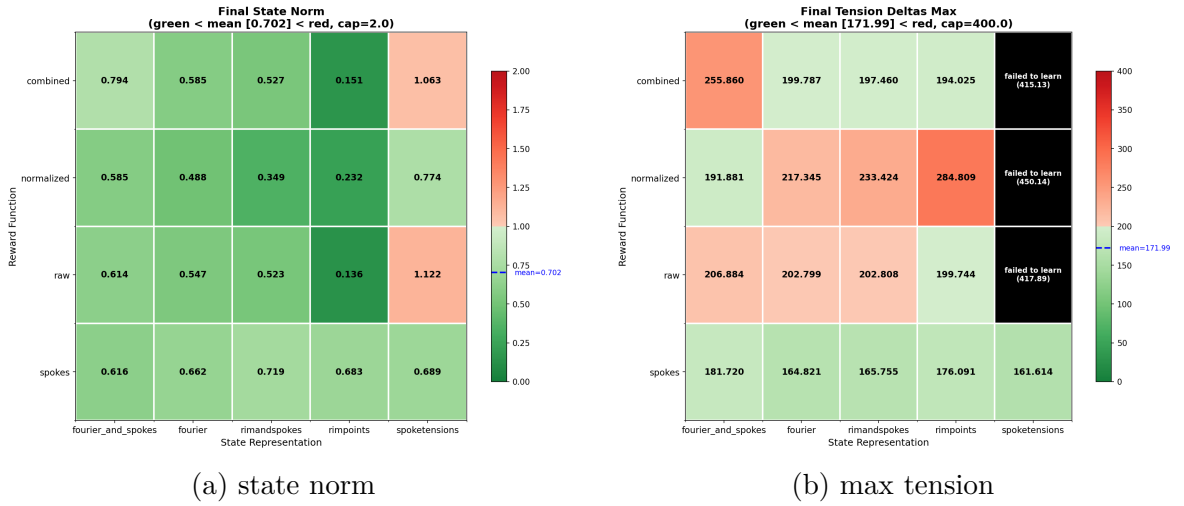


Figure 4.3: Rainbow

4.1.3 PPO

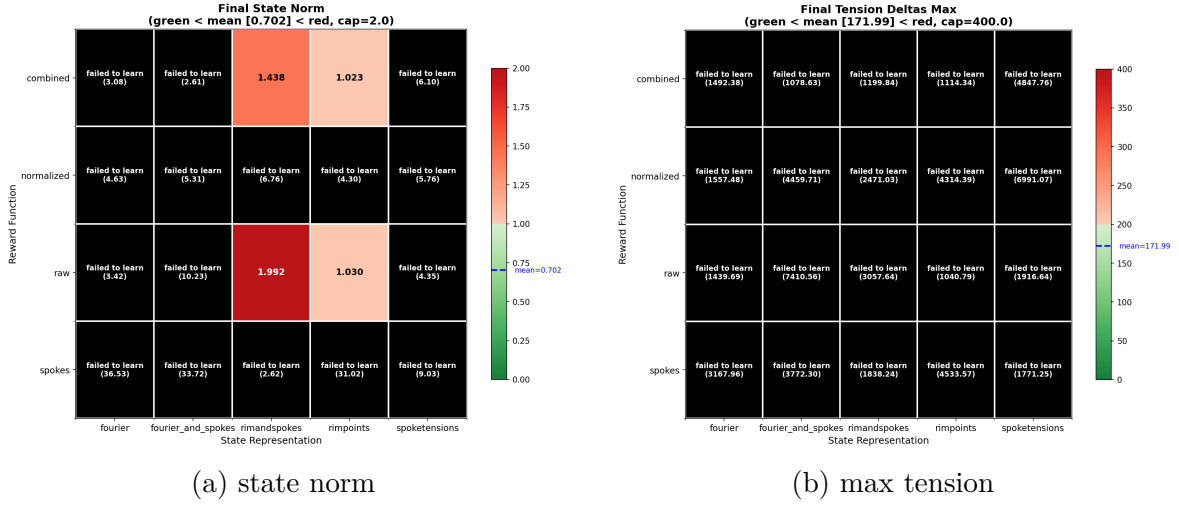


Figure 4.4: PPO continous

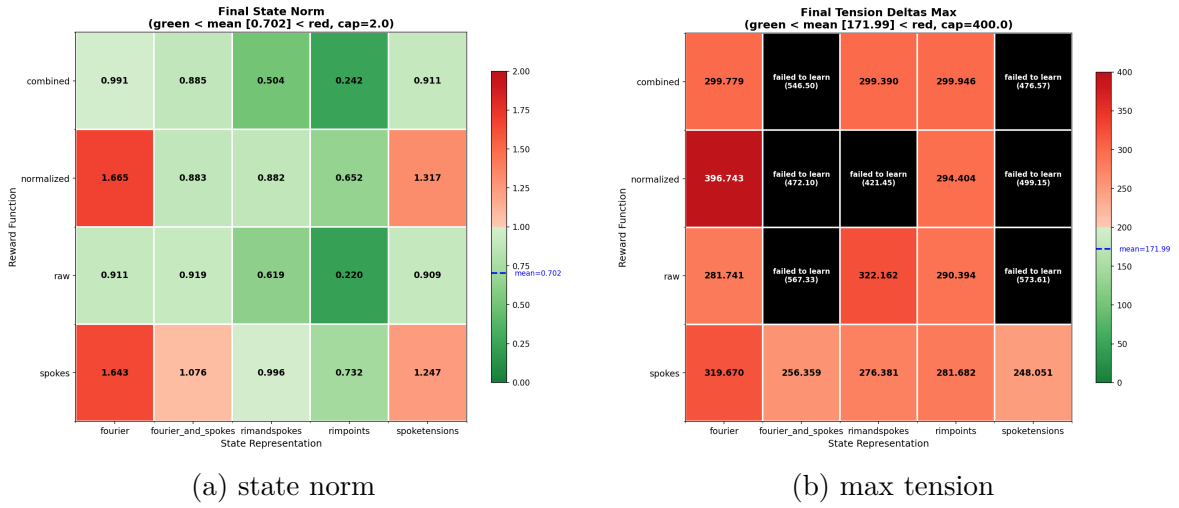


Figure 4.5: PPO discrete

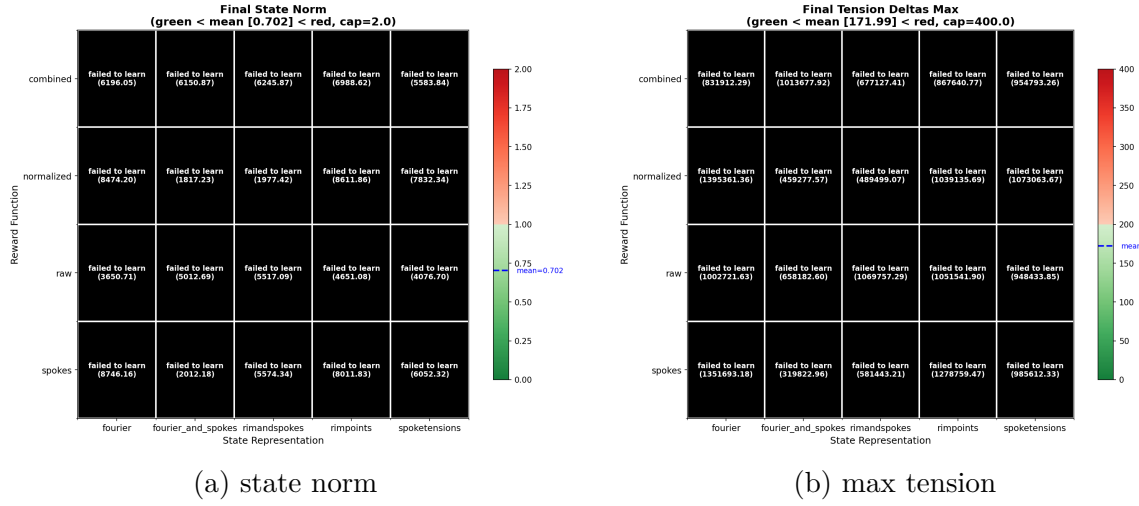


Figure 4.6: PPO all spokes

4.1.4 TD-MPC2

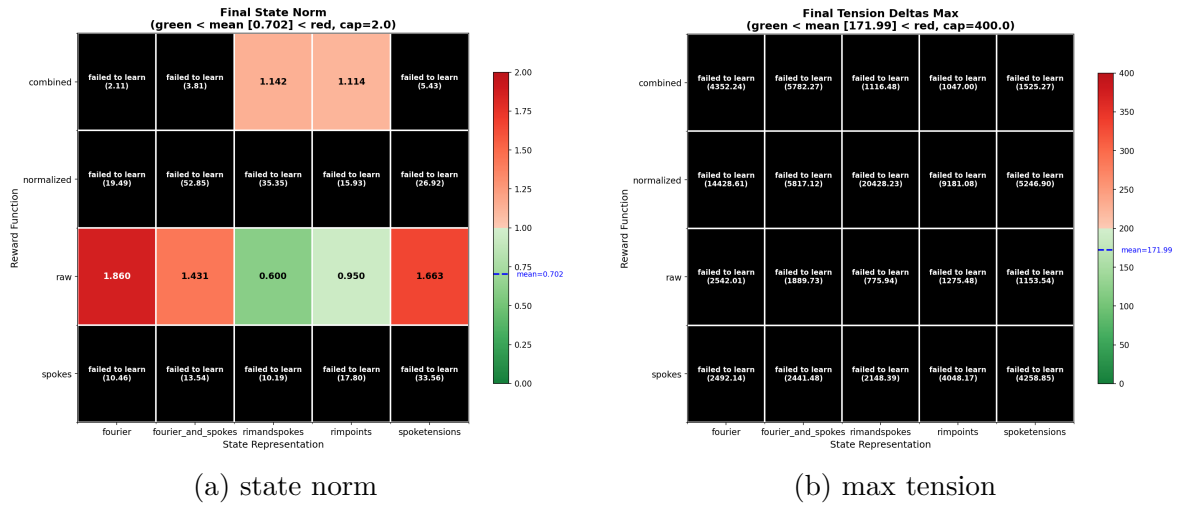


Figure 4.7: TDMPC all spokes

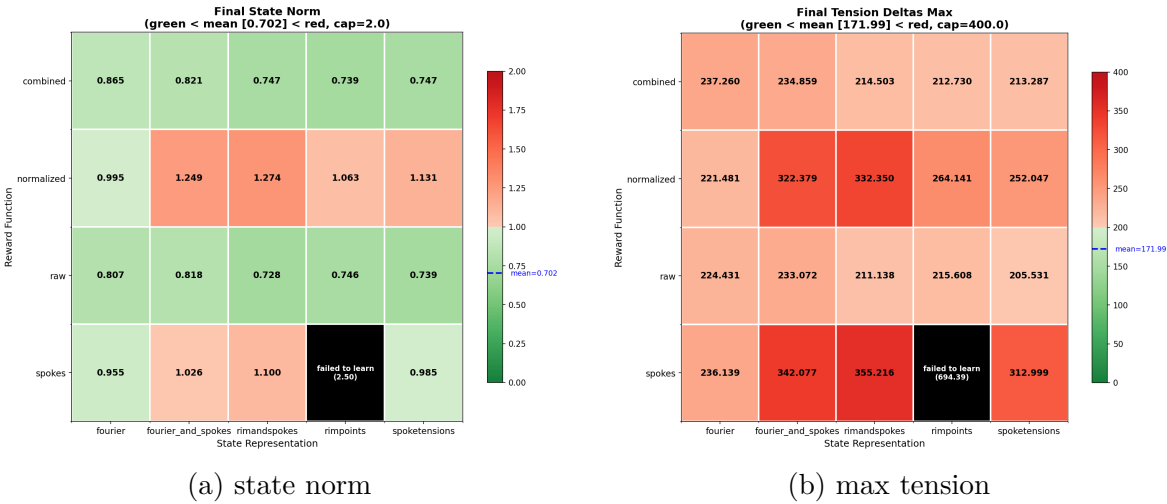


Figure 4.8: TDMPC continous

Chapter 5

Discussion and Conclusion

Bibliography

- [1] Matteo Hessel et al. “Rainbow: Combining Improvements in Deep Reinforcement Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (2018).
- [2] Matthew Ford. “Reinventing the Wheel: Stress Analysis, Stability, and Optimization of the Bicycle Wheel”. In: 2018. URL: <https://api.semanticscholar.org/CorpusID:216797463>.
- [3] Will Dabney et al. “Implicit Quantile Networks for Distributional Reinforcement Learning”. In: *International Conference on Machine Learning*. PMLR. 2018, pp. 1096–1105.
- [4] Hado Van Hasselt, Arthur Guez, and David Silver. “Deep Reinforcement Learning with Double Q-Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. 2016.
- [5] Ziyu Wang et al. “Dueling Network Architectures for Deep Reinforcement Learning”. In: *International Conference on Machine Learning*. PMLR. 2016, pp. 1995–2003.
- [6] Tom Schaul et al. “Prioritized Experience Replay”. In: *arXiv preprint arXiv:1511.05952* (2015).
- [7] Meire Fortunato et al. “Noisy Networks for Exploration”. In: *International Conference on Learning Representations*. 2018.
- [8] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *arXiv preprint arXiv:1506.02438* (2015).
- [9] Nicklas Hansen, Hao Su, and Xiaolong Wang. “TD-MPC2: Scalable, Robust World Models for Continuous Control”. In: *International Conference on Learning Representations* (2024).
- [10] Diganta Misra. “Mish: A Self Regularized Non-Monotonic Activation Function”. In: *arXiv preprint arXiv:1908.08681* (2019).

Appendix

.1 Code Listings

Extended code listings that are too long for the main text.